

Figure 7: THS on, normal memory compaction contiguity CDF.

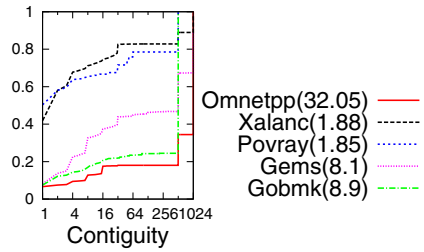


Figure 8: THS on, normal memory compaction contiguity CDF.

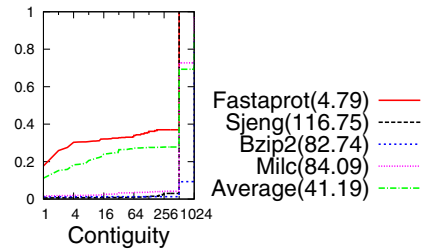


Figure 9: THS on, normal memory compaction contiguity CDF.

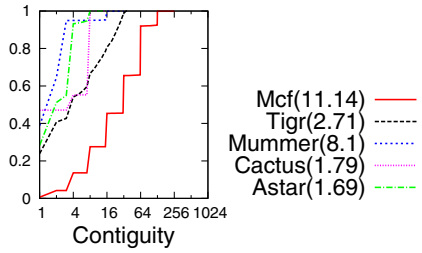


Figure 10: THS off, normal memory compaction contiguity CDF.

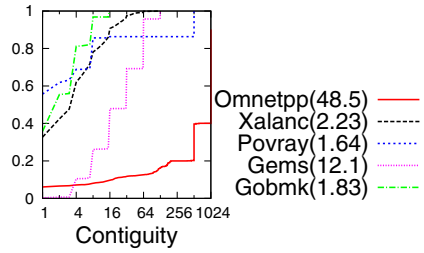


Figure 11: THS off, normal memory compaction contiguity CDF.

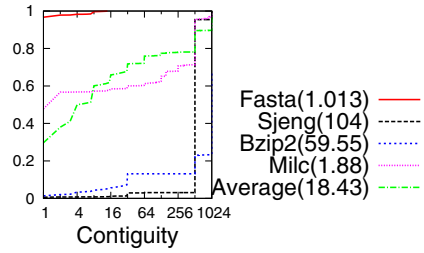


Figure 12: THS off, normal memory compaction contiguity CDF.

6.3. Superpaging, Low Memory Compaction

Figures 13, 14, and 15 present a worst-case scenario setting for Linux, where THS is turned off and memory compaction is greatly reduced via disabling the `defrag` kernel flag. While *no* kernel uses or recommends this setting, we study it to ensure that sufficient contiguity exists, even when there are almost no mechanisms to explicitly generate it. In fact, our results show that on average, contiguity drops only marginally compared to the THS off, normal memory compaction case to 15 pages on average. While important benchmarks like `Mcf`, `Mummer`, and `Omnetpp` do lose compared to the prior settings, they retain sufficiently high intermediate contiguity. For example, even though `Mummer's` average contiguity is now 1.3, roughly 50% of its 4KB pages enjoy 4-page contiguity. Correctly exploiting this gives our TLBs a 4× reach.

6.4. Superpaging, Memory Compaction, Memhog

We now focus on the impact of system load on fragmentation and contiguity. Figure 16 shows how contiguity is affected when `memhog` runs with each benchmark and fragments 25% and 50% of system memory. Our studies have shown that combined with the other running system processes, `memhog` with 50% heavily fragments almost all memory and causes page fault rates to greatly increase. We assume default Linux settings (THS enabled, normal memory compaction).

One might initially expect higher load to lower contiguity. Surprisingly, we find the *opposite* trend to hold when using `memhog(25%)`, with contiguity rising from 41 to roughly 43 pages on average. For some benchmarks, the gain is markedly high; for example, `Mcf` and `GemsFDTFD` contiguities are boosted by an order of magnitude. The primary reason for this is that higher load invokes the memory compaction daemon more often. This in turn provides the buddy allocator more contiguous physical blocks.

Greatly fragmenting the system with `memhog(50%)` however, does reduce contiguity. However, even this intermediate contiguity is relatively high, averaging close to 10 pages. For heavy TLB-pressure benchmarks like `Mcf` and `Mummer`, this configuration still achieves higher contiguity than without system load. As such, the buddy allocator, in tandem with memory compaction, manages to actually leverage the additional load to increase contiguity.

6.5. No Superpaging, Memory Compaction, Memhog

This represents the scenario where THS is turned off despite high system load. While kernel settings would not typically allow this, we use this setting to stress-test our measurements. We find that even under the pessimistic setting of no THS and `memhog(50%)`, the average contiguity is above 5. TLB coalescing can thus potentially provide a 5× reach.

6.6. Summary of Results

Three primary conclusions can be drawn from our real-system characterizations. First, under *every* single configuration, even those that are unrealistically severe, the buddy allocator, compaction daemon, and THS support succeed in inadvertently generating great intermediate contiguity. Second, system load can have surprising implications on contiguity, often increasing it. For some benchmarks that suffer from high TLB misses, such as `Mcf`, this is a promising observation. Third, superpages are ill-equipped to handle this contiguity. Therefore, coalescing techniques to harness this intermediate contiguity are warranted.

7. CoLT Evaluations

We now evaluate CoLT's benefits, focusing on per-application miss rate reductions and performance gains.

7.1. TLB Miss Rate Analysis

7.1.1. CoLT TLB miss rates. Figure 18 quantifies CoLT's TLB miss reductions. Benchmarks are ordered from highest to lowest TLB miss rates. We first capture the number of L1 and L2 TLB misses for a baseline configuration with 32-entry and 128-entry L1 and L2 TLBs (4-way) and a 16-entry superpage TLB. Note that we count misses for both the set-associative L1 TLB and the superpage TLB as L1 TLB misses since they are checked in parallel and have